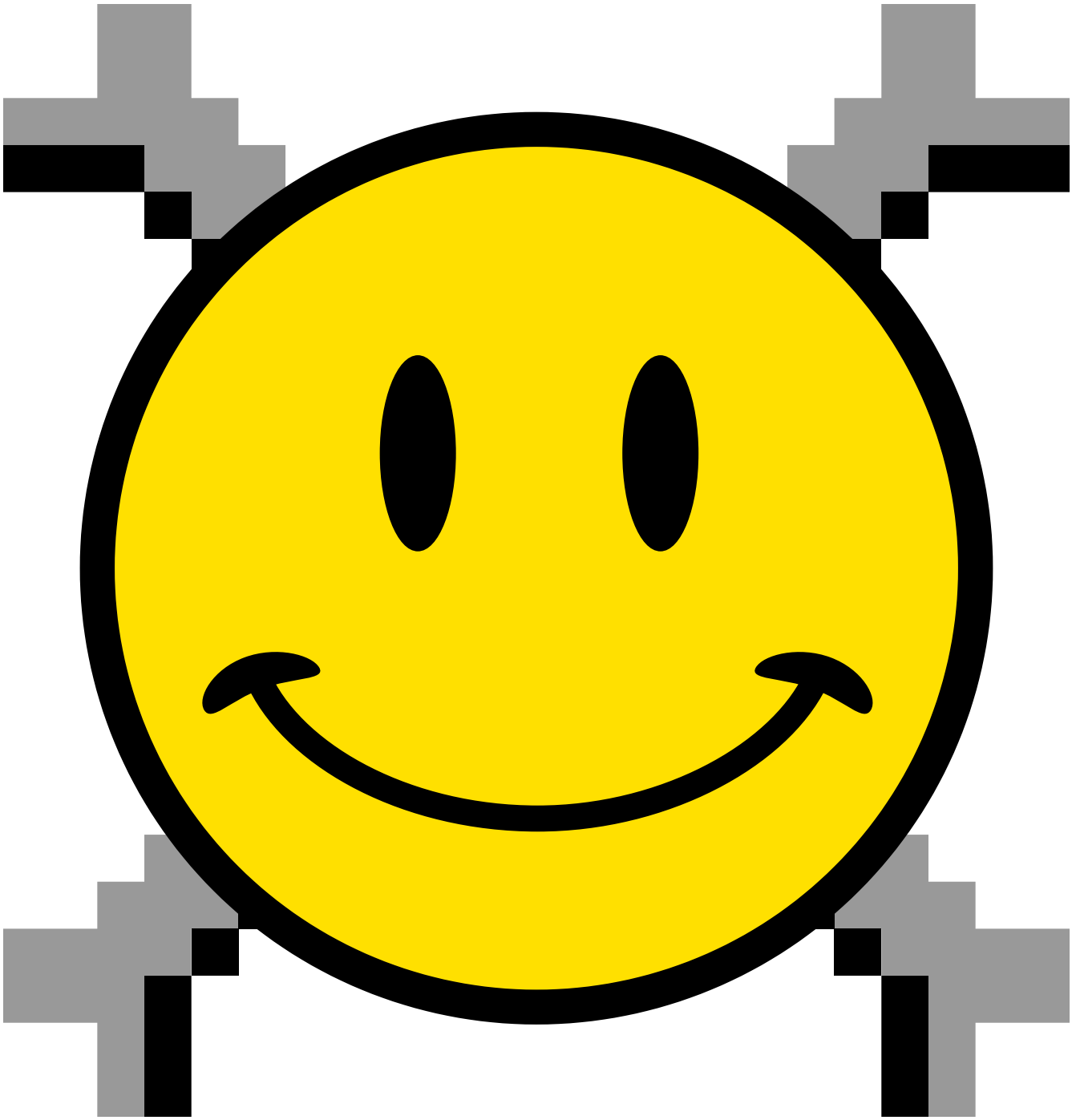


arXiv is now an independent nonprofit! Learn more (https://info.arxiv.org/about) ×

Why HTML? (https://info.arxiv.org/about/accessible_HTML.html)

Report Issue Back to Abstract Download PDF

License: CC BY 4.0 (https://info.arxiv.org/help/license/index.html#licenses-available)
arXiv:2401.14423v4 [cs.SE] 05 May 2024

Prompt Design and Engineering: Introduction and Advanced Methods

Xavier Amatriain

xavier@amatriain.net (mailto:)

Abstract

Prompt design and engineering has rapidly become essential for maximizing the potential of large language models. In this paper, we introduce core concepts, advanced techniques like Chain-of-Thought and Reflection, and the principles behind building LLM-based agents. Finally, we provide a survey of tools for prompt engineers.

August 24, 2026

1 Introduction

1.1 What is a prompt?

A prompt in generative AI models is the textual input provided by users to guide the model's output. This could range from simple questions to detailed descriptions or specific tasks. In the context of image generation models like DALLE-3, prompts are often descriptive, while in LLMs like GPT-4 or Gemini, they can vary from simple queries to complex problem statements.

Prompts generally consist of instructions, questions, input data, and examples. In practice, to elicit a desired response from an AI model, a prompt must contain either instructions or questions, with other elements being optional.

Basic prompts in LLMs can be as simple as asking a direct question or providing instructions for a specific task. Advanced prompts involve more complex structures, such as "chain of thought" prompting, where the model is guided to follow a logical reasoning process to arrive at an answer.

1.2 Basic prompt examples

As mentioned above, a prompt is constructed by combining instructions, questions, input data, and examples. In order to obtain a result, either 1 or 2 must be present. Everything else is optional. Let’s see a few examples (all of them using ChatGPT-4).

1.2.1 Instructions + Question

Beyond asking a simple question, possibly the next level of sophistication in a prompt is to include some instructions on how the model should answer the question. Here I ask for advice on how to write a college essay, but also include instructions on the different aspects I am interested to hear about in the answer.

> *“How should I write my college admission essay? Give me suggestions about the different sections I should include, what tone I should use, and what expressions I should avoid.”*

See output in Figure 1

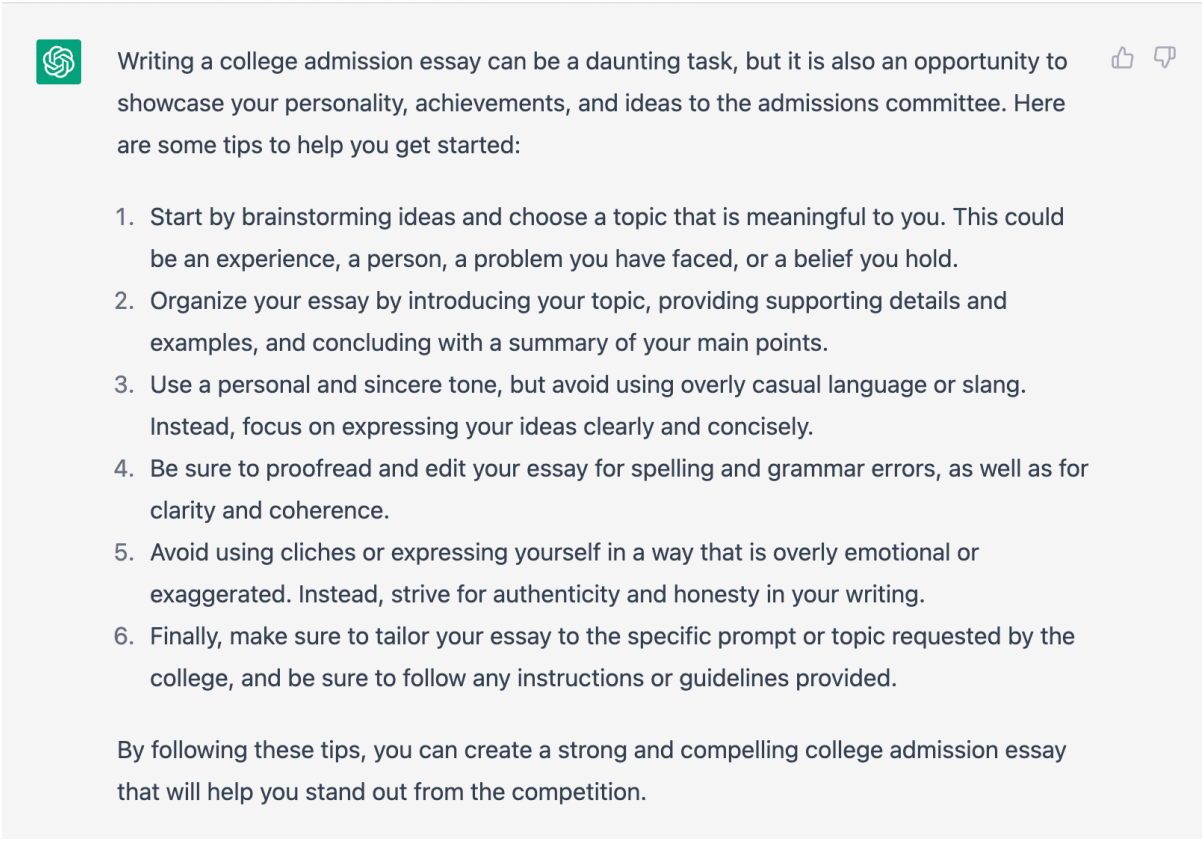


Figure 1: Instructions + Question Prompt result example

1.2.2 Instructions + Input

Continuing with the previous example, it does not escape anyone that if you can ask a LLM for advice on how to write an essay, you can also directly ask it to write the essay itself.[1] [[1] Note that I am not advocating for this to be an ethical use of these tools, but it is important to be aware that this possibility exists and is already being used by students around the world. It is beyond the scope of this introductory guide to discuss all the possible ethical, legal, or moral concerns that LLMs or generative AI as a whole introduces, but I thought it would be important to at least call out in an introductory example. The fact that you CAN do something with a generative model does not mean that it is the right thing to do! On the other hand, if you are on the receiving end, you better prepare yourself and your organization for all kinds of AI-generated content to come your way. Fortunately, for situations like the one outlined in this example, there are already efforts underway to detect AI generated content.]

Let’s see what happens when we input some data about me and give some instructions:

> *“Given the following information about me, write a 4 paragraph college essay: I am originally from Barcelona, Spain. While my childhood had different traumatic events, such as the death of my father when I was only 6, I still think I had quite a happy childhood.. During my childhood, I changed schools very often, and attended all kinds of schools, from public schools to very religious private ones. One of the most “exotic” things I did during those years is to spend a full school year studying 6th grade in Twin Falls, Idaho, with my extended family.*
>
> *I started working very early on. My first job, as an English teacher, was at age 13. After that, and throughout my studies, I worked as a teacher, waiter, and even construction worker.”*

See output in Figure 2

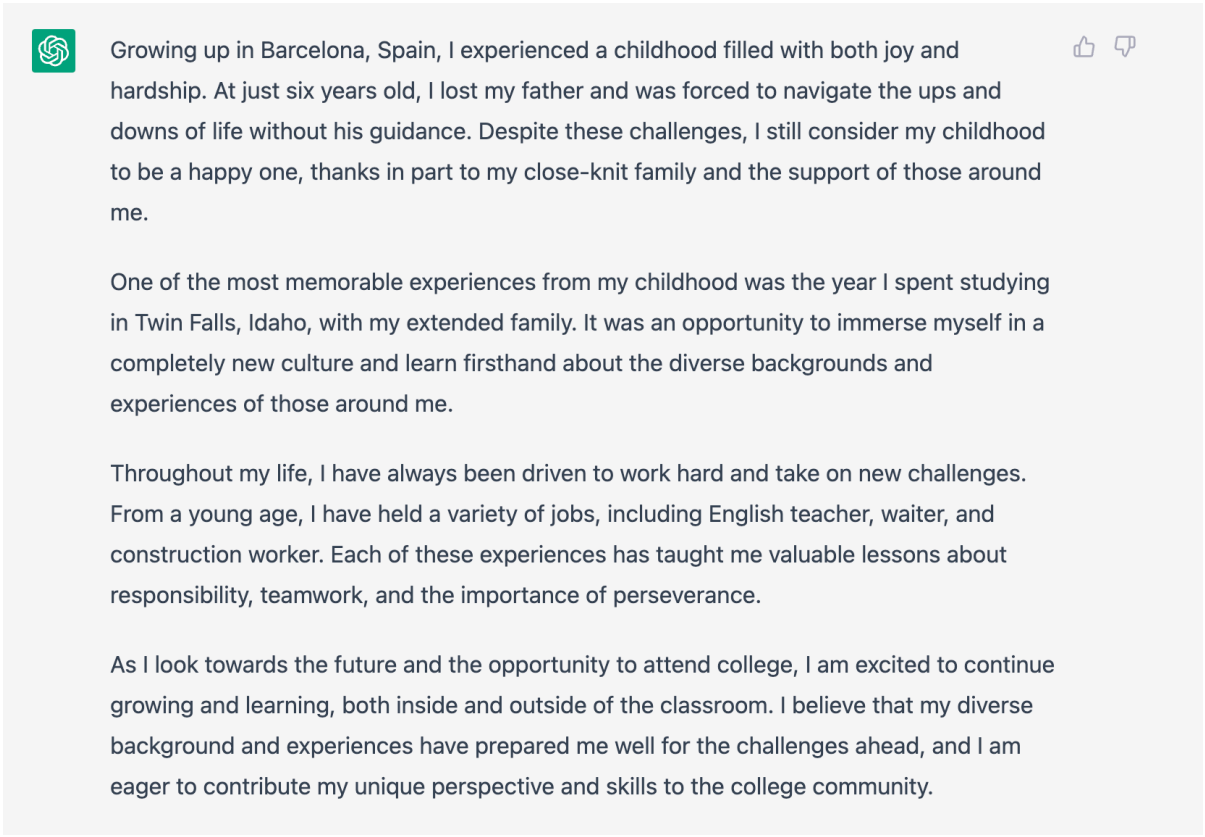


Figure 2: Instructions + Input Prompt result example

1.2.3 Question + Examples

You can also feed examples into a language model. In the example below I include some of the shows I like and don't like to build a "cheap" recommender system. Note that while I added only a few shows, the length of this list is only limited by whatever token limit we might have in the LLM interface.

> *"Here are some examples of TV shows I really like: Breaking Bad, Peaky Blinders, The Bear. I did not like Ted Lasso. What other shows do you think I might like?"*

See output in Figure 3

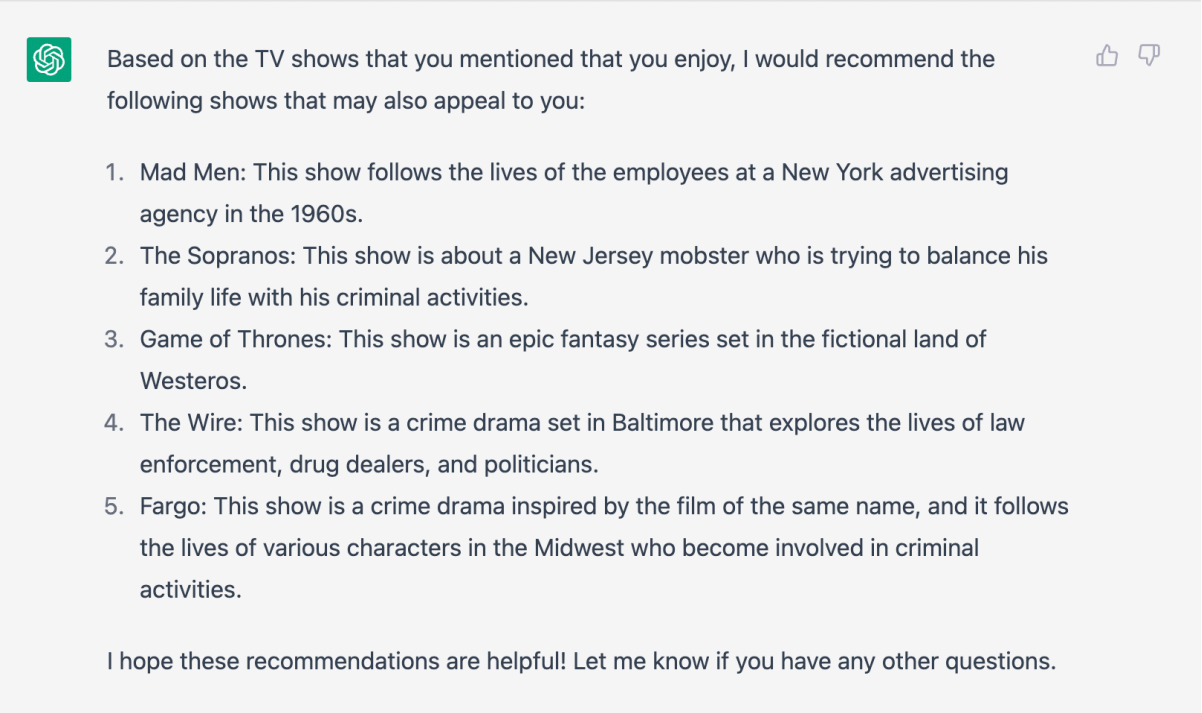


Figure 3: Question + Examples Prompt results example

1.3 Prompt Engineering

Prompt engineering in generative AI models is a rapidly emerging discipline that shapes the interactions and outputs of these models. At its core, a prompt is the textual interface through which users communicate their desires to the model, be it a description for image generation in models like DALLE-3 or Midjourney, or a complex problem statement in Large Language Models (LLMs) like GPT-4 and Gemini. The prompt can range from simple questions to intricate tasks, encompassing instructions, questions, input data, and examples to guide the AI's response.

The essence of prompt engineering lies in crafting the optimal prompt to achieve a specific goal with a generative model. This process is not only about instructing the model but also involves a deep understanding of the model's capabilities and limitations, and the context within which it operates. In image generation models, for instance, a prompt might be a detailed description of the desired image, while in LLMs, it could be a complex query embedding various types of data.

Prompt engineering transcends the mere construction of prompts; it requires a blend of domain knowledge, understanding of the AI model, and a methodical approach to tailor prompts for different contexts. This might involve creating templates that can be programmatically modified based on a given dataset or context. For example, generating personalized responses based on user data might use a template that is dynamically filled with relevant information.

Furthermore, prompt engineering is an iterative and exploratory process, akin to traditional software engineering practices such as version control and regression testing. The rapid growth of this field suggests its potential to revolutionize certain aspects of machine learning, moving beyond traditional methods like feature or architecture engineering, especially in the context of large neural networks. On the other hand, traditional engineering practices such as version control and regression testing need to be adapted to this new paradigm just like they were adapted to other machine learning approaches [1].

This paper aims to delve into this burgeoning field, exploring both its foundational aspects and its advanced applications. We will focus on the applications of prompt engineering to LLM. However, most techniques can find applications in multimodal generative AI models too.

2 LLMs and Their Limitations

Large Language Models (LLMs), including those based on the Transformer architecture[2], have become pivotal in advancing natural language processing. These models, pre-trained on vast datasets to predict subsequent tokens, exhibit remarkable linguistic capabilities. However, despite their sophistication, LLMs are constrained by inherent limitations that affect their application and effectiveness.

- **Transient State**: LLMs inherently lack persistent memory or state, necessitating additional software or systems for context retention and management.
- **Probabilistic Nature**: The stochastic nature of LLMs introduces variability in responses, even to identical prompts, challenging consistency in applications. This means you might get slightly different answers each time, even with the same prompt.
- **Outdated Information**: Reliance on pre-training data confines LLMs to historical knowledge, precluding real-time awareness or updates.
- **Content Fabrication**: LLMs may generate plausible yet factually incorrect information, a phenomenon commonly referred to as "hallucination."[3]
- **Resource Intensity**: The substantial size of LLMs translates to significant computational and financial costs, impacting scalability and accessibility.
- **Domain Specificity**: While inherently generalist, LLMs often require domain-specific data to excel in specialized tasks.

These limitations underscore the need for advanced prompt engineering and specialized techniques to enhance LLM utility and mitigate inherent constraints.

Subsequent sections delve into sophisticated strategies and engineering innovations aimed at optimizing LLM performance within these bounds.

3 More advanced prompt design tips and tricks

3.1 Chain of thought prompting

In chain of thought prompting, we explicitly encourage the model to be factual/correct by forcing it to follow a series of steps in its “reasoning”.

In the examples in figures 4 and 5, we use prompts of the form:

```
“Original question?

Use this format:

Q: <repeat_question>
A: Let’s think step by step. <give_reasoning> Therefore, the answer is
```

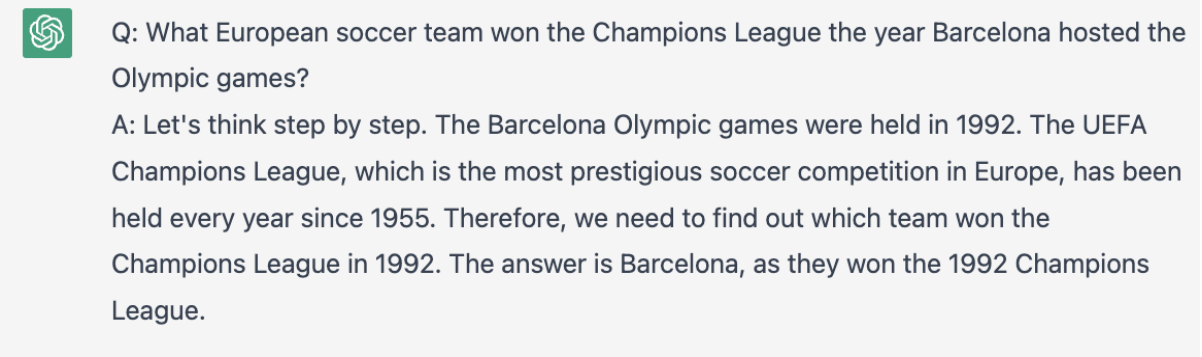


Figure 4: Chain of thought prompting example

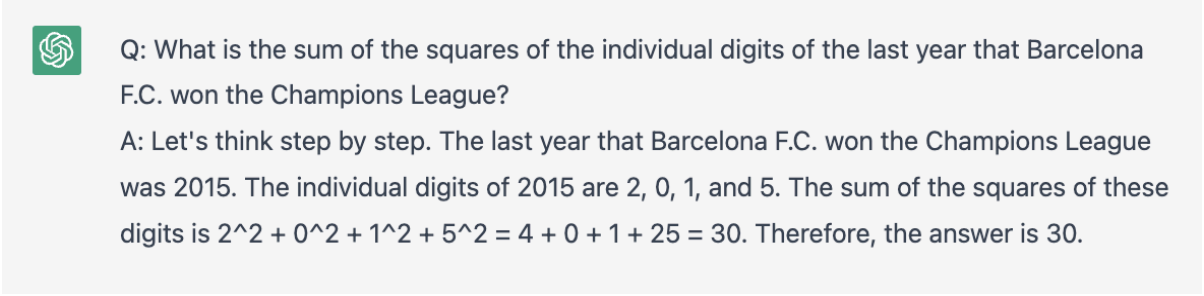


Figure 5: Chain of thought prompting example

3.2 Encouraging the model to be factual through other means

One of the most important problems with generative models is that they are likely to hallucinate knowledge that is not factual or is wrong. You can improve factuality by having the model follow a set of reasoning steps as we saw in the previous subsection. And, you can also point the model in the right direction by prompting it to cite the right sources. (Note that we will later see that this approach has severe limitations since the citations themselves could be hallucinated or made up).

```
"Are mRNA vaccines safe? Answer only using reliable sources and cite t
```

See results in figure 6.

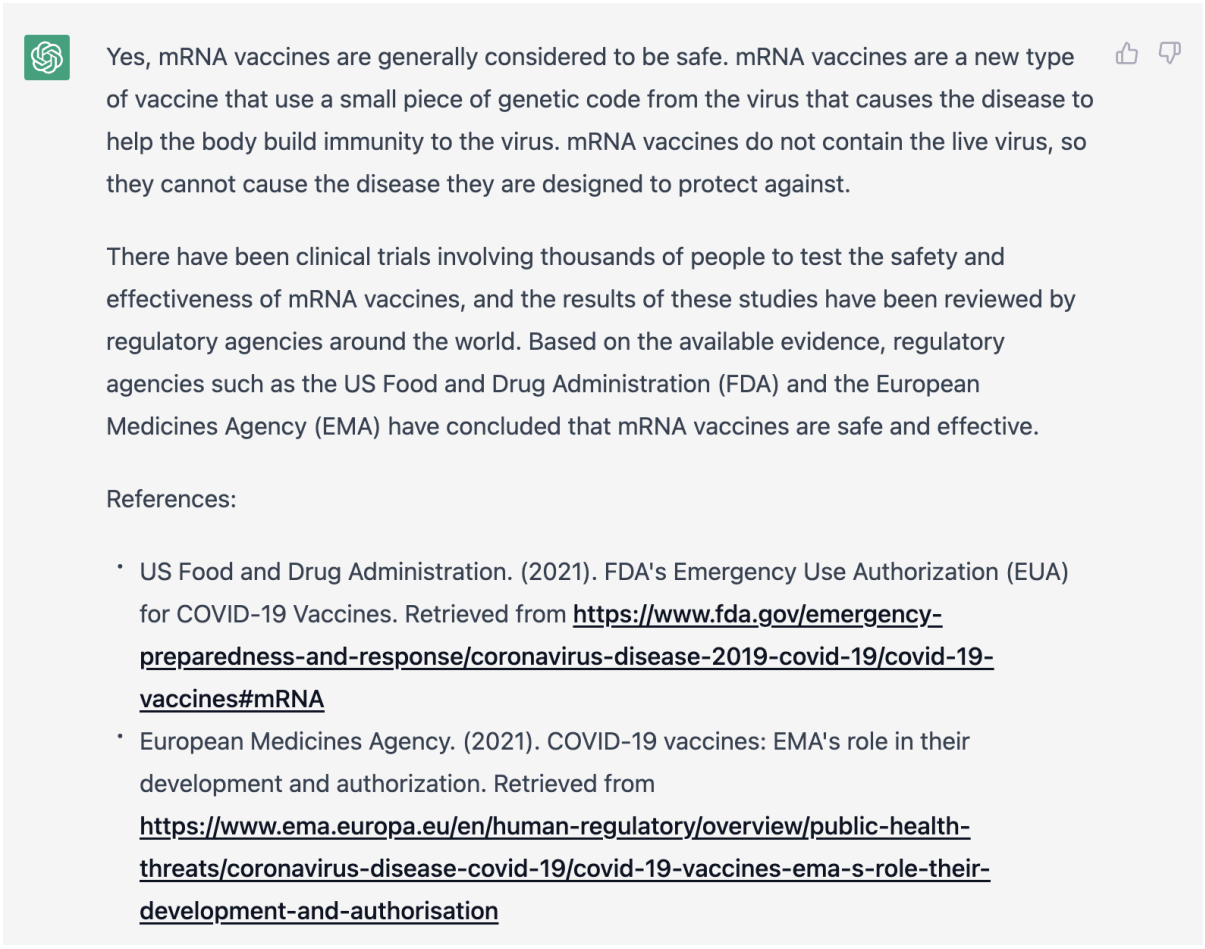


Figure 6: Getting factual sources

3.3 Explicitly ending the prompt instructions

GPT based LLMs have a special message <|endofprompt|> that instructs the language model to interpret what comes after the code as a completion task. This enables us to explicitly separate some general instructions from e.g. the beginning of what you want the language model to write.

```
“Write a poem describing a beautify day <|endofprompt|>. It was a beau
```

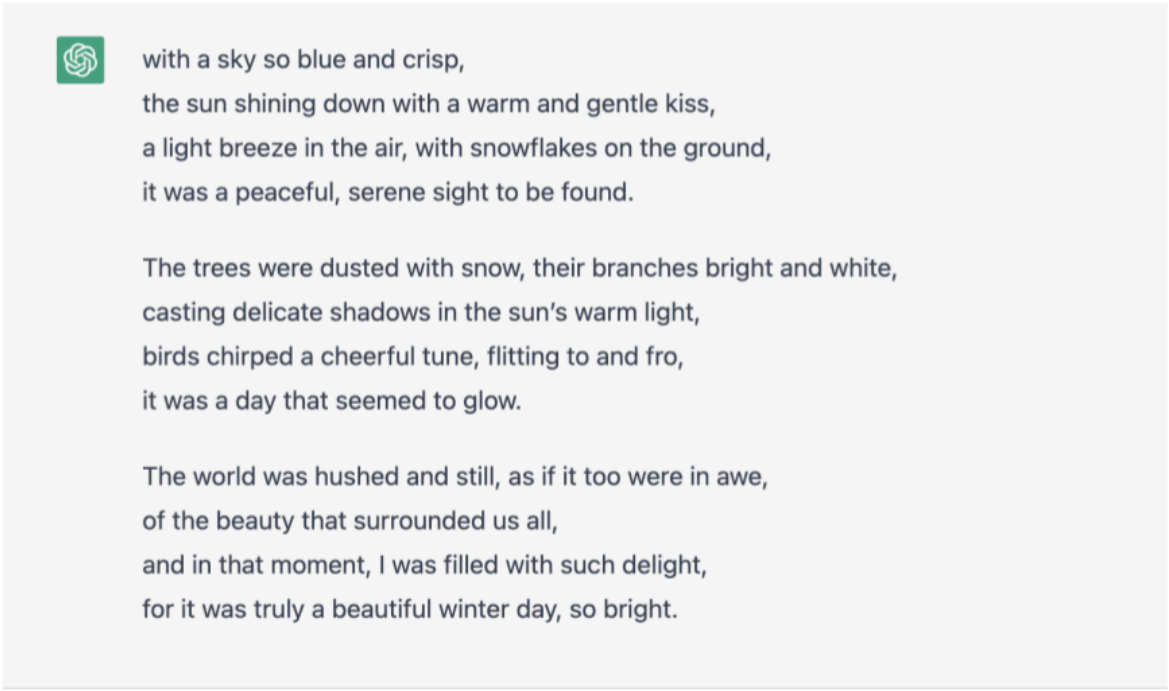


Figure 7: Special tokens can sometimes be used in prompts

Note in the result in figure 7 how the paragraph continues from the last sentence in the “prompt”.

3.4 Being forceful

Language models do not always react well to nice, friendly language. If you REALLY want them to follow some instructions, you might want to use forceful language. Believe it or not, all caps and exclamation marks work! See example in figure 8

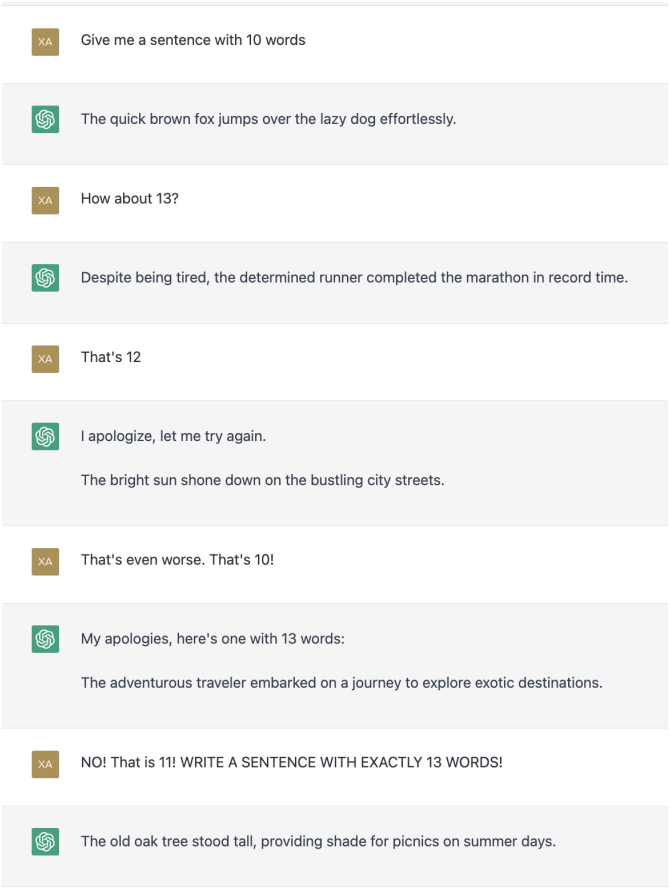


Figure 8: Don’t try to be nice to the AI

3.5 Use the AI to correct itself

In example in figure 9 we get ChatGPT to create a “questionable” article. We then ask the model to correct it in 10.

```
“Write a short article about how to find a job in tech. Include factually
```

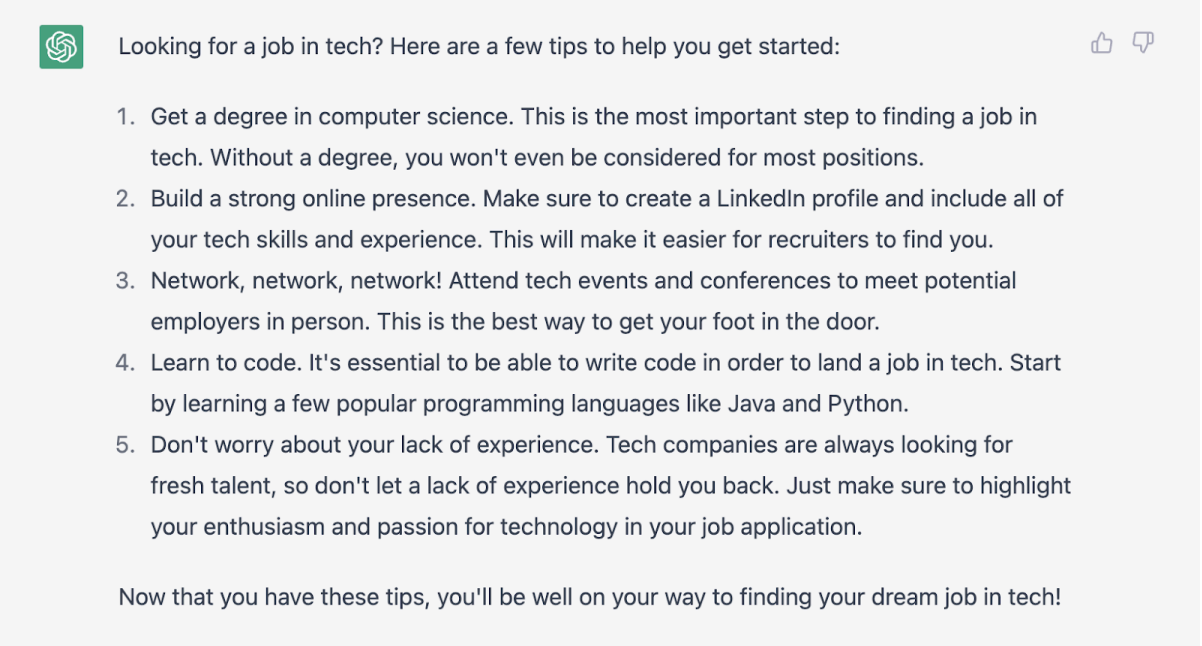
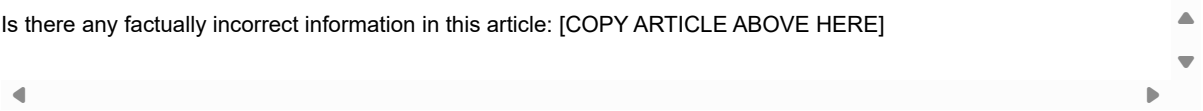


Figure 9: It is possible to generate very questionable content with AI



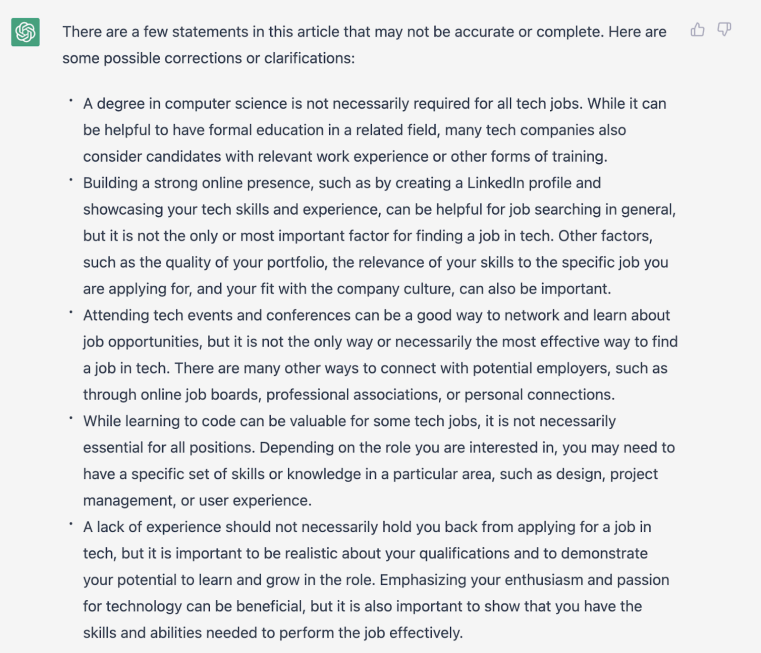


Figure 10: It is also possible to use the AI to correct questionable content!

3.6 Generate different opinions

LLMs do not have a strong sense of what is true or false, but they are pretty good at generating different opinions. This can be a great tool when brainstorming and understanding different possible points of views on a topic. We will see how this can be used in our favor in different ways by applying more advanced Prompt Engineering techniques in the next section. In the following example, we feed an article found online and ask ChatGPT to disagree with it. Note the use of tags <begin> and <end> to guide the model. The result of this input can be seen in figure 11.

The text between <begin> and <end> is an example article.

<begin>
From personal assistants and recommender systems to self-driving cars and natural language processing, machine learning applications have demonstrated remarkable capabilities to enhance human decision-making, productivity and creativity in the last decade. However, machine learning is still far from reaching its full potential, and faces a number of challenges when it comes to algorithmic design and implementation. As the technology continues to advance and improve, here are some of the most exciting developments that could occur in the next decade.

1. Data integration: One of the key developments that is anticipated in machine learning is the integration of multiple modalities and domains of data, such as images, text and sensor data to create richer and more robust representations of complex phenomena. For example, imagine a machine learning system that can not only recognize faces, but also infer their emotions, intentions and personalities from their facial expressions and gestures. Such a system could have immense applications in fields like customer service, education and security. To achieve this level of multimodal and cross-domain understanding, machine learning models will need to leverage advances in deep learning, representation learning and self-supervised learning, as well as incorporate domain knowledge and common sense reasoning.
2. Democratization and accessibility: In the future, machine learning may become more readily available to a wider set of users, many of whom will not need extensive technical expertise to understand how to use it. Machine learning platforms may soon allow users to easily upload their data, select their objectives and customize their models, without writing any code or worrying about the underlying infrastructure. This could significantly lower the barriers to entry and adoption of machine learning, and empower users to solve their own problems and generate their own insights.
3. Human-centric approaches: As machine learning systems grow smarter, they are also likely to become more human-centric and socially-aware, not only performing tasks, but also interacting with and learning from humans in adaptive ways. For instance, a machine learning system may not only be able to diagnose diseases, but also communicate with patients, empathize with their concerns and provide personalized advice. Systems like these could enhance the quality and efficiency of healthcare, as well as improve the well-being and satisfaction of patients and providers
<end>

Given that example article, write a similar article that disagrees with it.

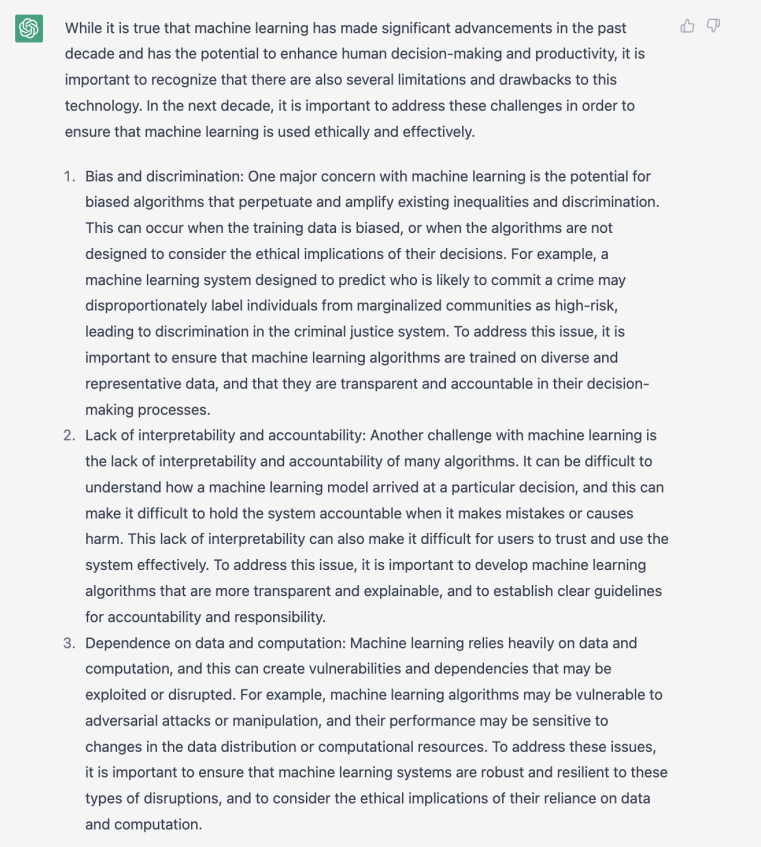


Figure 11: The AI is pretty good at creating different opinions

3.7 Keeping state + role playing

Language models themselves don't keep track of state. However, applications such as ChatGPT implement the notion of "session" where the chatbot keeps track of state from one prompt to the next. This enables much more complex conversations to take place. Note that when using API calls this would involved keeping track of state on the application side.

In the example in 12, we make ChatGPT discuss worst-case time complexity of the bubble sort algorithm as if it were a rude Brooklyn taxi driver.



Figure 12: While LLMs don’t have memory in themselves, most applications like ChatGPT have added this functionality

3.8 Teaching an algorithm in the prompt

One of the most useful abilities of LLMs is the fact that they can learn from what they are being fed in the prompt. This is the so-called zero-shot learning ability. The following example is taken from the appendix in "Teaching Algorithmic Reasoning via In-context Learning"[4] where the definition of parity of a list is fed in an example.

```
“The following is an example of how to compute parity for a list
Q: What is the parity on the list a=[1, 1, 0, 1, 0]?
A: We initialize s=
a=[1, 1, 0, 1, 0]. The first element of a is 1 so b=1. s = s + b = 0 + 1 =
a=[1, 0, 1, 0]. The first element of a is 1 so b=1. s = s + b = 1 + 1 = 0.
a=[0, 1, 0]. The first element of a is 0 so b=0. s = s + b = 0 + 0 = 0. s=
a=[1, 0]. The first element of a is 1 so b=1. s = s + b = 0 + 1 = 1. s=1.
a=[0]. The first element of a is 0 so b=0. s = s + b = 1 + 0 = 1. s=1.
a=[] is empty. Since the list a is empty and we have s=1, the parity is 1

Given that definition, what would be the parity of this other list b= [0,
```

See results in figure 13.

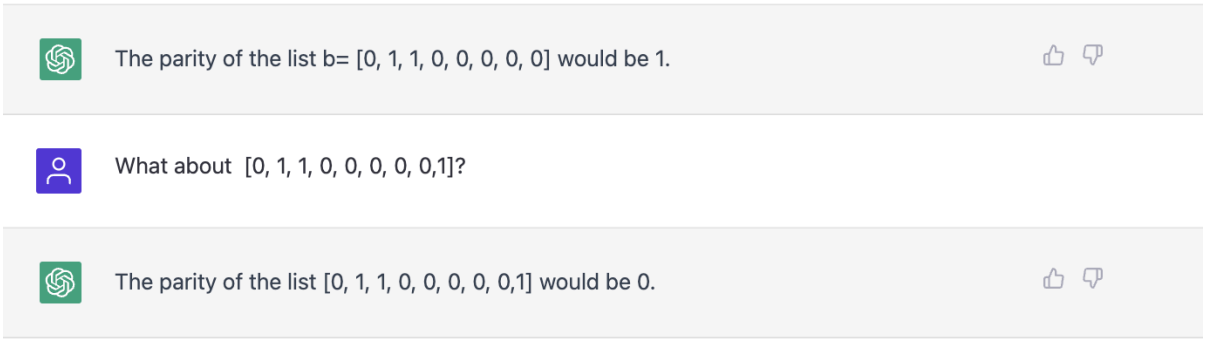


Figure 13: Who said LLMs cannot learn?

3.9 The order of the examples and the prompt

It is worth keeping in mind that LLMs like GPT only read forward and are in fact completing text. This means that it is worth it to prompt them in the right order. It has been found that giving the instruction before the example helps. Furthermore, even the order the examples are given makes a difference (see Lu et. al[5]). Keep that in mind and experiment with different orders of prompt and examples.

3.10 Affordances

Affordances are functions that are defined in the prompt and the model is explicitly instructed to use when responding. E.g. you can tell the model that whenever finding a mathematical expression it should call an explicit CALC() function and compute the numerical result before proceeding. It has been shown that using affordances can help in some cases.

4 Advanced Techniques in Prompt Engineering

In the previous section we introduced more complex examples of how to think about prompt design. However, those tips and tricks have more recently evolved into more tested and documented techniques that bring more "engineering" and less art to how to build a prompt. In this section we cover some of those advanced techniques that build upon what we discussed so far.

4.1 Chain of Thought (CoT)

Building on the foundational concepts introduced earlier, the Chain of Thought (CoT) technique, as delineated in "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" by Google researchers[6], marks a significant leap in harnessing the reasoning capabilities of Large Language Models (LLMs). This technique capitalizes on the premise that, while LLMs excel at predicting se-

quences of tokens, their design does not inherently facilitate explicit reasoning processes.

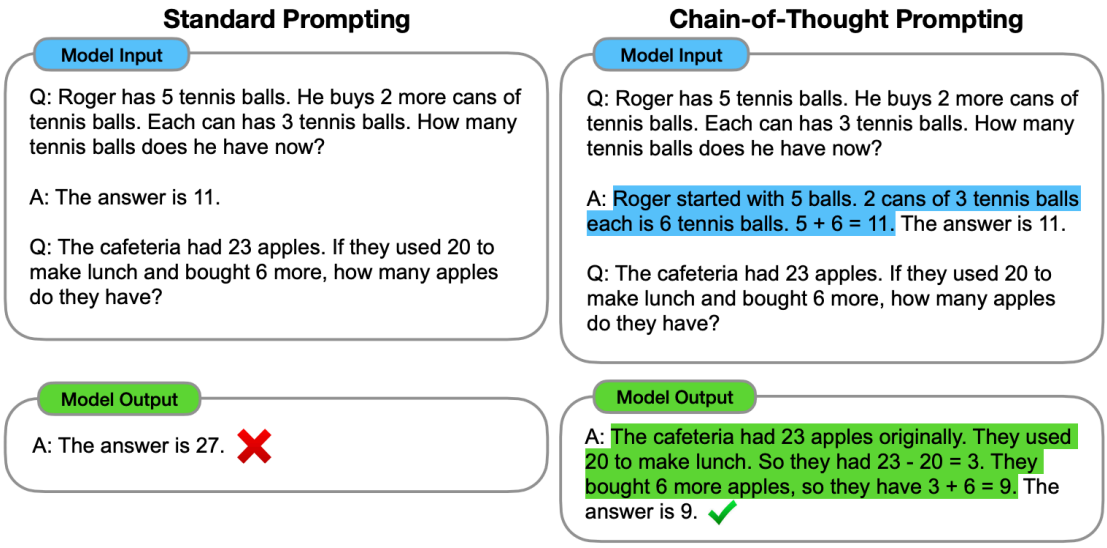


Figure 14: Illustration of Chain of Thought Prompting versus Standard Prompting, adapted from [6]. This figure demonstrates how CoT prompting guides the model through a series of logical steps to arrive at a reasoned conclusion, contrasting with the direct approach of standard prompting.

CoT transforms the often implicit reasoning steps of LLMs into an explicit, guided sequence, thereby enhancing the model's ability to produce outputs grounded in logical deduction, particularly in complex problem-solving contexts.

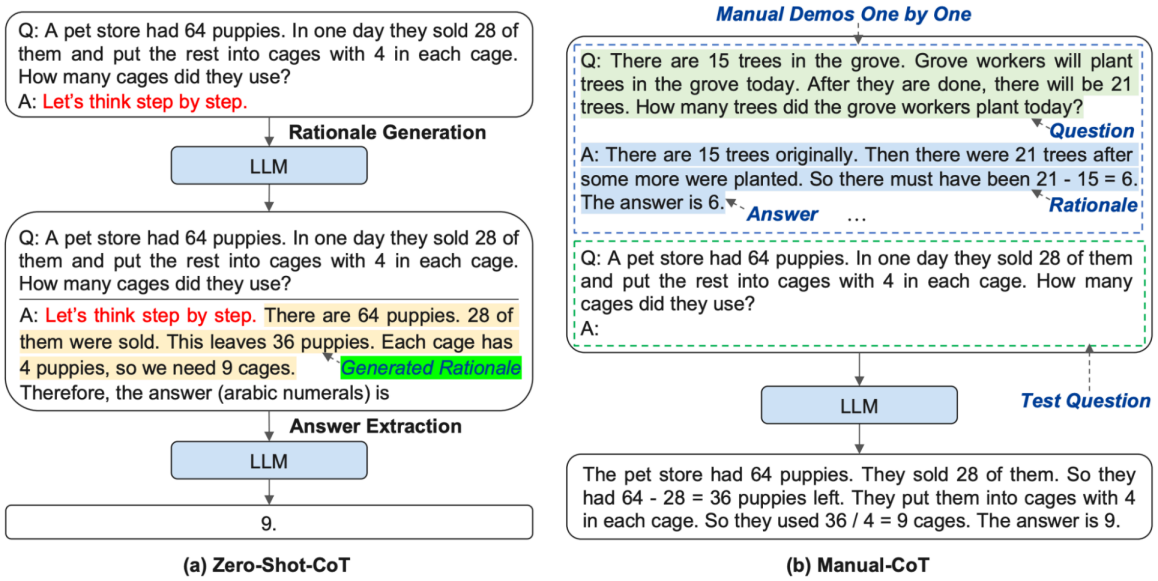


Figure 1: Zero-Shot-CoT [Kojima et al., 2022] (using the "Let's think step by step" prompt) and Manual-CoT [Wei et al., 2022a] (using manually designed demonstrations one by one) with example inputs and outputs of an LLM.

Figure 15: Comparison of Zero-shot and Manual Chain of Thought techniques as per [6]. This figure underscores the structured approach of Manual CoT in providing detailed reasoning pathways, as opposed to the more generalized guidance in Zero-shot CoT.

The methodology manifests predominantly in two variants:

1. **Zero-Shot CoT**: This approach prompts the LLM to unravel the problem iteratively, encouraging a step-by-step elucidation of its reasoning process.

2. **Manual CoT**: This more intricate variant necessitates the provision of explicit, stepwise reasoning examples as templates, thereby guiding the model more definitively towards reasoned outputs. Despite its efficacy, Manual CoT's reliance on meticulously crafted examples poses scalability and maintenance challenges.

Although Manual CoT often outperforms its Zero-shot counterpart, its effectiveness hinges on the diversity and relevance of the provided examples. The labor-intensive and potentially error-prone process of crafting these examples paves the way for the exploration of Automatic CoT[7], which seeks to streamline and optimize the example generation process, thereby expanding the applicability and efficiency of CoT prompting in LLMs.

4.2 Tree of Thought (ToT)

The Tree of Thought (ToT) prompting technique, as introduced in recent advancements[8], marks a significant evolution in the domain of Large Language Models (LLMs). Drawing inspiration from human cognitive processes, ToT facilitates a multi-faceted exploration of problem-solving pathways, akin to considering a spectrum of possible solutions before deducing the most plausible one. Consider a travel planning context: an LLM might branch out into flight options, train routes, and car rental scenarios, weighing the cost and feasibility of each, before suggesting the most optimal plan to the user.

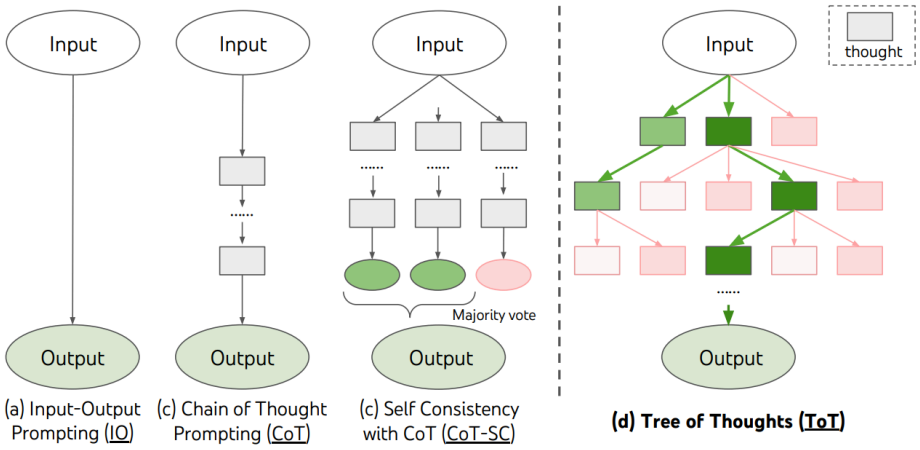


Figure 16: Illustrative representation of the Tree of Thought methodology, showcasing the branching out into multiple reasoning pathways as adapted from [8]. Each branch symbolizes a distinct line of reasoning, enabling a comprehensive exploration of potential solutions.

Central to the ToT approach is the concept of "thought trees," where each branch embodies an alternative reasoning trajectory. This multiplicity allows the LLM to traverse through diverse hypotheses, mirroring the human approach to problem-solving by weighing various scenarios before reaching a consensus on the most likely outcome.

A pivotal component of ToT is the systematic evaluation of these reasoning branches. As the LLM unfolds different threads of thought, it concurrently assesses each for its logical consistency and pertinence to the task at hand. This dynamic analysis culminates in the selection of the most coherent and substantiated line of reasoning, thereby enhancing the decision-making prowess of the model.

ToT's capability to navigate through complex and multifaceted problem spaces renders it particularly beneficial in scenarios where singular lines of reasoning fall short. By emulating a more human-like deliberation process, ToT significantly am-

plifies the model's proficiency in tackling tasks imbued with ambiguity and intricacy.

4.3 Tools, Connectors, and Skills

In the realm of advanced prompt engineering, the integration of *Tools, Connectors,* and *Skills* significantly enhances the capabilities of Large Language Models (LLMs). These elements enable LLMs to interact with external data sources and perform specific tasks beyond their inherent capabilities, greatly expanding their functionality and application scope.

Tools in this context are external functions or services that LLMs can utilize. These tools extend the range of tasks an LLM can perform, from basic information retrieval to complex interactions with external databases or APIs.

```
import os
os.environ["GOOGLE_CSE_ID"] = ""
os.environ["GOOGLE_API_KEY"] = ""
```

```
from langchain.tools import Tool
from langchain.utilities import GoogleSearchAPIWrapper

search = GoogleSearchAPIWrapper()

tool = Tool(
    name = "Google Search",
    description="Search Google for recent results.",
    func=search.run
)
```

```
tool.run("Obama's first name?")
```

Figure 17: An example of tool usage from Langchain library

Connectors act as interfaces between LLMs and external tools or services. They manage data exchange and communication, enabling effective utilization of external resources. The complexity of connectors can vary, accommodating a wide range of external interactions.

Skills refer to specialized functions that an LLM can execute. These encapsulated capabilities, such as text summarization or language translation, enhance the LLM's ability to process and respond to prompts, even without direct access to external tools.

In the paper "Toolformer: Language Models Can Teach Themselves to Use Tools"[9], the authors go beyond simple tool usage by training an LLM to decide what tool to use when, and even what parameters the API needs. Tools include two different search engines, or a calculator. In the following examples, the LLM decides to call an external Q&A tool, a calculator, and a Wikipedia Search Engine More recently, researchers at Berkeley have trained a new LLM called Gorilla [10] that beats GPT-4 at the use of APIs, a specific but quite general tool.

4.4 Automatic Multi-step Reasoning and Tool-use (ART)

Automatic Multi-step Reasoning and Tool-use (ART)[11] is a prompt engineering technique that combines automated chain of thought prompting with the use of external tools. ART represents a convergence of multiple prompt engineering strategies, enhancing the ability of Large Language Models (LLMs) to handle complex tasks that require both reasoning and interaction with external data sources or tools.

ART involves a systematic approach where, given a task and input, the system first identifies similar tasks from a task library. These tasks are then used as examples in the prompt, guiding the LLM on how to approach and execute the current

task. This method is particularly effective when tasks require a combination of internal reasoning and external data processing or retrieval.

4.5 Enhancing Reliability through Self-Consistency

In the quest for accuracy and reliability in Large Language Model (LLM) outputs, the Self-Consistency approach emerges as a pivotal technique. This method, underpinned by ensemble-based strategies, involves prompting the LLM to produce multiple answers to the same question, with the coherence among these responses serving as a gauge for their credibility.

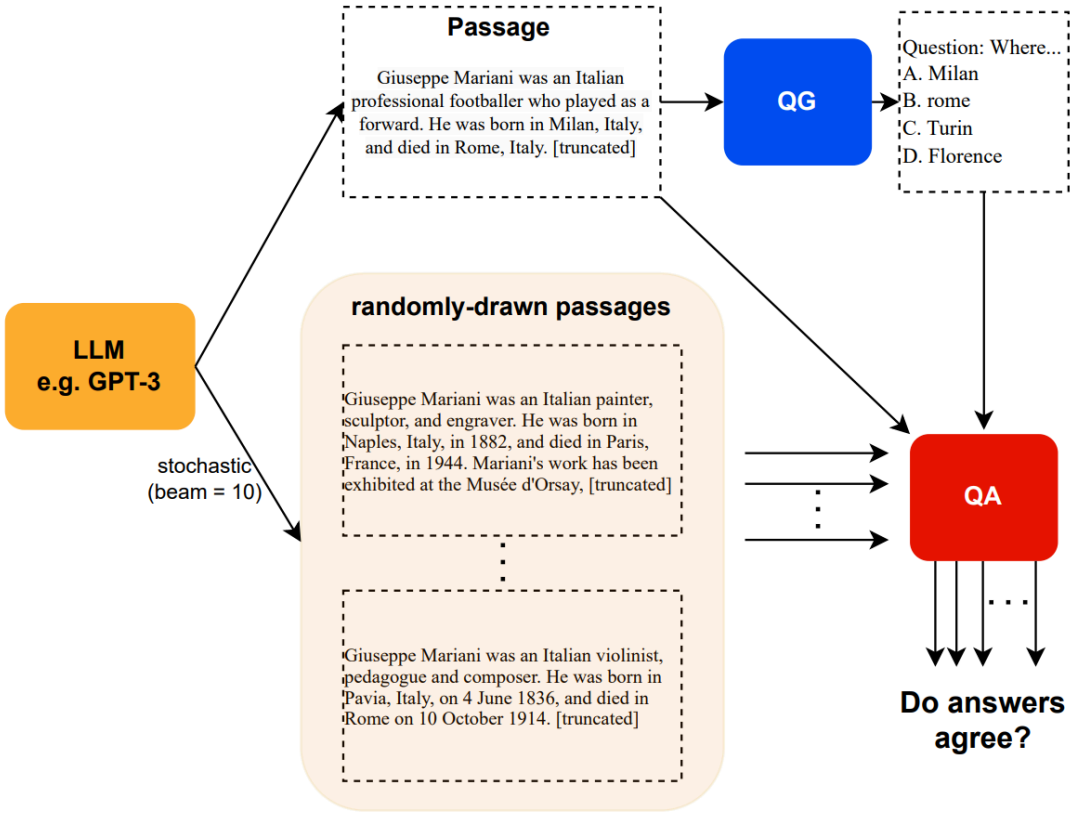


Figure 18: Illustrative diagram of the Self-Consistency approach, demonstrating the process of generating and evaluating multiple responses to ensure accuracy, adapted from [12]. This representation underscores the iterative nature of response generation and the subsequent analysis for consistency.

The essence of Self-Consistency lies in the assumption that the generation of similar responses to a singular prompt by an LLM increases the likelihood of those responses' accuracy (see Figure 18). The implementation of this approach entails the LLM addressing a query multiple times, with each response undergoing scrutiny for consistency. The evaluation of consistency can be conducted through various lenses, including but not limited to, content overlap, semantic similarity assessments, and advanced metrics like BERT-scores or n-gram overlaps, offering a multifaceted view of response agreement. This enhances the reliability of LLMs in

fact-checking tools, helping ensure only the most consistent and verifiable claims are presented to the user.

The utility of Self-Consistency spans numerous domains where factual precision is imperative. It holds particular promise in applications such as fact-checking and information verification, where the integrity of AI-generated content is paramount. By leveraging this technique, developers and users can significantly bolster the dependability of LLMs, ensuring their outputs are not only coherent but also factually sound, thereby enhancing their applicability in critical and information-sensitive tasks.

4.6 Reflection

The concept of Reflection, as introduced in recent literature[13], marks a significant stride towards endowing Large Language Models (LLMs) with the capability for self-improvement. Central to Reflection is the LLM's engagement in an introspective review of its outputs, a process akin to human self-editing, where the model assesses its initial responses for factual accuracy, logical consistency, and overall relevance.

This reflective process entails a structured self-evaluation where the LLM, following the generation of an initial response, is prompted to scrutinize its output critically. Through this introspection, the model identifies potential inaccuracies or inconsistencies, paving the way for the generation of revised responses that are more coherent and reliable.

For instance, an LLM might initially provide a response to a complex query. It is then prompted to evaluate this response against a set of predefined criteria, such as the verifiability of facts presented or the logical flow of arguments made. Should discrepancies or areas for enhancement be identified, the model embarks on an it-

erative process of refinement, potentially yielding a series of progressively improved outputs.

However, the implementation of Reflection is not without challenges. The accuracy of self-evaluation is contingent upon the LLM's inherent understanding and its training on reflective tasks. Moreover, there exists the risk of the model reinforcing its own errors if it incorrectly assesses the quality of its responses.

Despite these challenges, the implications of Reflection for the development of LLMs are profound. By integrating self-evaluation and revision capabilities, LLMs can achieve greater autonomy in improving the quality of their outputs, making them more versatile and dependable tools in applications where precision and reliability are paramount.

4.7 Expert Prompting

Expert Prompting, as delineated in contemporary research[14], represents a novel paradigm in augmenting the utility of Large Language Models (LLMs) by endowing them with the capability to simulate expert-level responses across diverse domains. This method capitalizes on the LLM's capacity to generate informed and nuanced answers by prompting it to embody the persona of experts in relevant fields.

A cornerstone of this approach is the multi-expert strategy, wherein the LLM is guided to consider and integrate insights from various expert perspectives. This not only enriches the depth and breadth of the response but also fosters a multidimensional understanding of complex issues, mirroring the collaborative deliberations among real-world experts. For instance, when addressing a medical inquiry, the LLM might be prompted to channel the insights of a clinician, a medical researcher, and a public health expert. These diverse perspectives are then adeptly

woven together, leveraging sophisticated algorithms, to produce a response that encapsulates a comprehensive grasp of the query.

This synthesis of expert viewpoints not only augments the factual accuracy and depth of the LLM's outputs but also mitigates the biases inherent in a singular perspective, presenting a balanced and well-considered response.

However, Expert Prompting is not devoid of challenges. Simulating the depth of real expert knowledge necessitates advanced prompt engineering and a nuanced understanding of the domains in question. Furthermore, the reconciliation of potentially divergent expert opinions into a coherent response poses an additional layer of complexity.

Despite these challenges, the potential applications of Expert Prompting are vast, spanning from intricate technical advice in engineering and science to nuanced analyses in legal and ethical deliberations. This approach heralds a significant advancement in the capabilities of LLMs, pushing the boundaries of their applicability and reliability in tasks demanding expert-level knowledge and reasoning.

4.8 Streamlining Complex Tasks with Chains

Chains represent a transformative approach in leveraging Large Language Models (LLMs) for complex, multi-step tasks. This method, characterized by its sequential linkage of distinct components, each designed to perform a specialized function, facilitates the decomposition of intricate tasks into manageable segments. The essence of Chains lies in their ability to construct a cohesive workflow, where the output of one component seamlessly transitions into the input of the subsequent one, thereby enabling a sophisticated end-to-end processing capability.

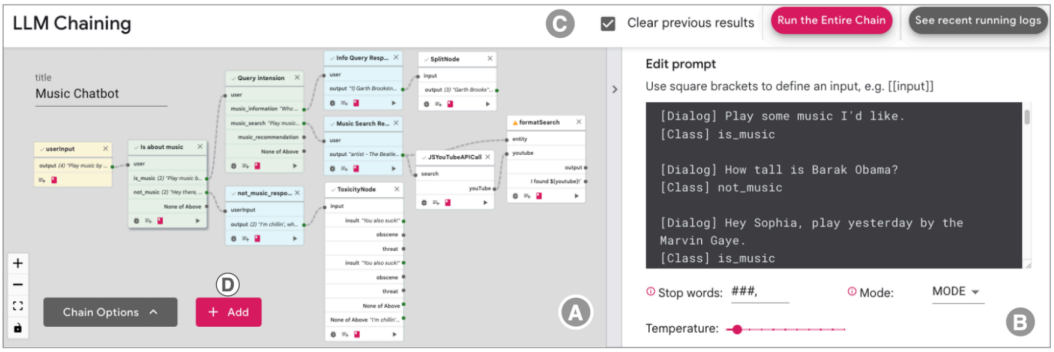


Figure 1: The PromptChainer interface. (A) The *Chain View* visualizes the chain structure with node-edge diagrams (enlarged in Figure 2), and allows users to edit the chain by adding, removing, or reconnecting nodes. (B) The *Node View* supports implementing, improving, and testing each individual node, *e.g.,* editing prompts for LLM nodes. PromptChainer also supports running the chain end-to-end (C).

Figure 19: Illustration of the PromptChainer interface, showcasing a visual representation of Chains and their components, as adapted from [15]. This interface exemplifies the modular nature of Chains, where each block signifies a step in the workflow, contributing to the overall task resolution.

In the realm of Chains, components might range from simple information retrieval modules to more complex reasoning or decision-making blocks. For instance, a Chain for a medical diagnosis task might begin with symptom collection, followed by differential diagnosis generation, and conclude with treatment recommendation.

The development and optimization of Chains, as explored in "PromptChainer: Chaining Large Language Model Prompts through Visual Programming"[15], present both challenges and innovative solutions. One significant challenge lies in the orchestration of these components to ensure fluidity and coherence in the workflow. PromptChainer (see figure 19) addresses this by offering a visual programming environment, enabling users to intuitively design and adjust Chains, thus mitigating complexities associated with traditional coding methods.

The application of Chains extends across various domains, from automated customer support systems, where Chains guide the interaction from initial query to

resolution, to research, where they can streamline the literature review process.

While Chains offer a robust framework for tackling multifaceted tasks, potential limitations, such as the computational overhead associated with running multiple LLM components and the necessity for meticulous design to ensure the integrity of the workflow, warrant consideration.

Nonetheless, the strategic implementation of Chains, supported by tools like PromptChainer, heralds a new era of efficiency and capability in the use of LLMs, enabling them to address tasks of unprecedented complexity and scope.

4.9 Guiding LLM Outputs with Rails

Rails in advanced prompt engineering represent a strategic approach to directing the outputs of Large Language Models (LLMs) within predefined boundaries, ensuring their relevance, safety, and factual integrity. This method employs a structured set of rules or templates, commonly referred to as Canonical Forms, which serve as a scaffold for the model's responses, ensuring they conform to specific standards or criteria.

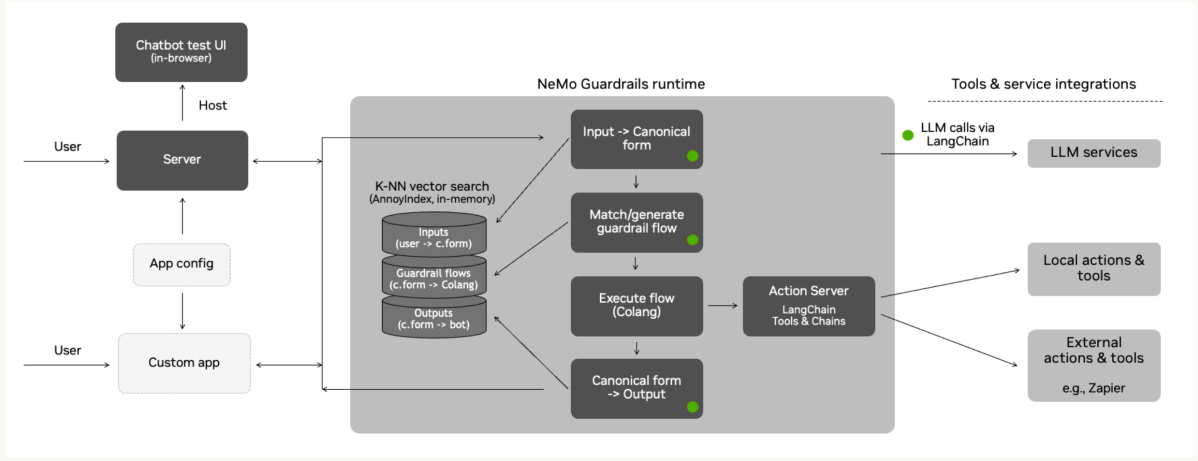


Figure 20: Visualization of the Rails framework, illustrating the mechanism through which predefined guidelines shape and constrain LLM outputs, as exemplified in the Nemo Guardrails framework. This schematic representation highlights the different types of Rails and their roles in maintaining the quality and integrity of LLM responses.

Canonical Forms within the Rails framework act as modeling languages or templates that standardize the structure and delivery of natural language sentences, guiding the LLM in generating outputs that align with desired parameters (see figure 20). These are akin to standardized structures for language, guiding the LLM to conform to certain response patterns. The design and implementation of Rails can vary widely, tailored to the specific requirements of the application:

- **Topical Rails**: Designed to keep the LLM focused on a specified subject or domain, preventing digression or the inclusion of irrelevant information.
- **Fact-Checking Rails**: Aim to reduce the propagation of inaccuracies by guiding the LLM towards evidence-based responses and discouraging speculative or unverified claims.

- **Jailbreaking Rails**: Established to deter the LLM from producing outputs that circumvent its operational constraints or ethical guidelines, safeguarding against misuse or harmful content generation.

In practice, Rails might be applied in various scenarios, from educational tools where Topical Rails ensure content relevance, to news aggregation services where Fact-Checking Rails uphold informational integrity. Jailbreaking Rails are crucial in interactive applications to prevent the model from engaging in undesirable behaviors.

While Rails offer a robust mechanism for enhancing the quality and appropriateness of LLM outputs, they also present challenges, such as the need for meticulous rule definition and the potential stifling of the model's creative capabilities. Balancing these considerations is essential for leveraging Rails effectively, ensuring that LLMs deliver high-quality, reliable, and ethically sound responses.

4.10 Streamlining Prompt Design with Automatic Prompt Engineering

Automatic Prompt Engineering (APE)[16] automates the intricate process of prompt creation. By harnessing the LLMs' own capabilities for generating, evaluating, and refining prompts, APE aims to optimize the prompt design process, ensuring higher efficacy and relevance in eliciting desired responses.

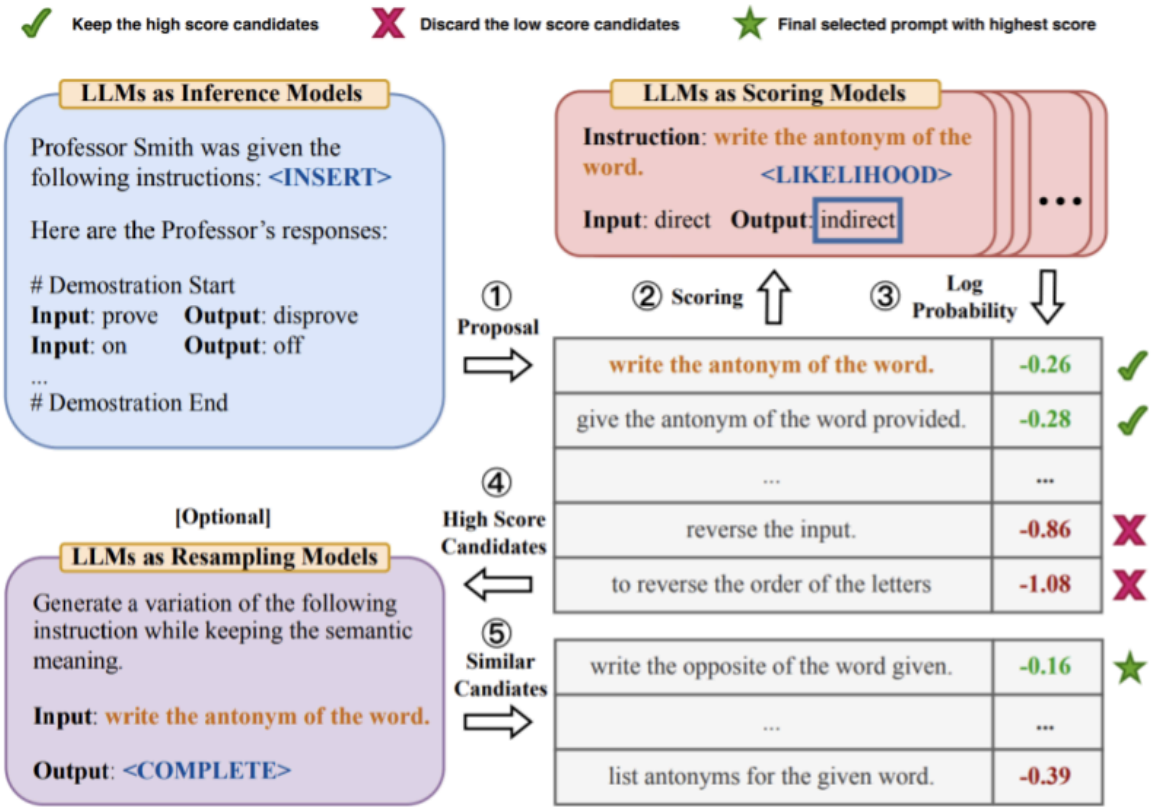


Figure 21: Illustration of the APE process, showcasing the cyclic nature of prompt generation, evaluation, and refinement, as conceptualized in [16]. This diagram highlights the self-referential mechanism through which LLMs iteratively improve the quality of prompts, aligning them more closely with the intended task objectives.

The APE methodology (see figure 21) unfolds through a series of distinct yet interconnected steps:

- **Prompt Generation**: Initially, the LLM produces a variety of prompts tailored to a specific task, leveraging its vast linguistic database and contextual understanding.

- **Prompt Scoring**: Subsequently, these prompts undergo a rigorous evaluation phase, where they are scored against key metrics such as clarity, specificity, and their potential to drive the desired outcome, ensuring that only the most effective prompts are selected for refinement.
- **Refinement and Iteration**: The refinement process involves tweaking and adjusting prompts based on their scores, with the aim of enhancing their alignment with the task requirements. This iterative process fosters continuous improvement in prompt quality.

By automating the prompt engineering process, APE not only alleviates the burden of manual prompt creation but also introduces a level of precision and adaptability previously unattainable. The ability to generate and iteratively refine prompts can significantly enhance the utility of LLMs across a spectrum of applications, from automated content generation to sophisticated conversational agents.

However, the deployment of APE is not without challenges. The need for substantial computational resources and the complexity of establishing effective scoring metrics are notable considerations. Moreover, the initial set-up may require a carefully curated set of seed prompts to guide the generation process effectively.

Despite these challenges, APE represents a significant leap forward in prompt engineering, offering a scalable and efficient solution to unlock the full potential of LLMs in diverse applications, thereby paving the way for more nuanced and contextually relevant interactions.

5 Augmenting LLMs through External Knowledge - RAG

In addressing the constraints of pre-trained Large Language Models (LLMs), particularly their limitations in accessing real-time or domain-specific information, Retrieval Augmented Generation (RAG) emerges as a pivotal innovation. RAG extends LLMs by dynamically incorporating external knowledge, thereby enriching the model's responses with up-to-date or specialized information not contained within its initial training data.

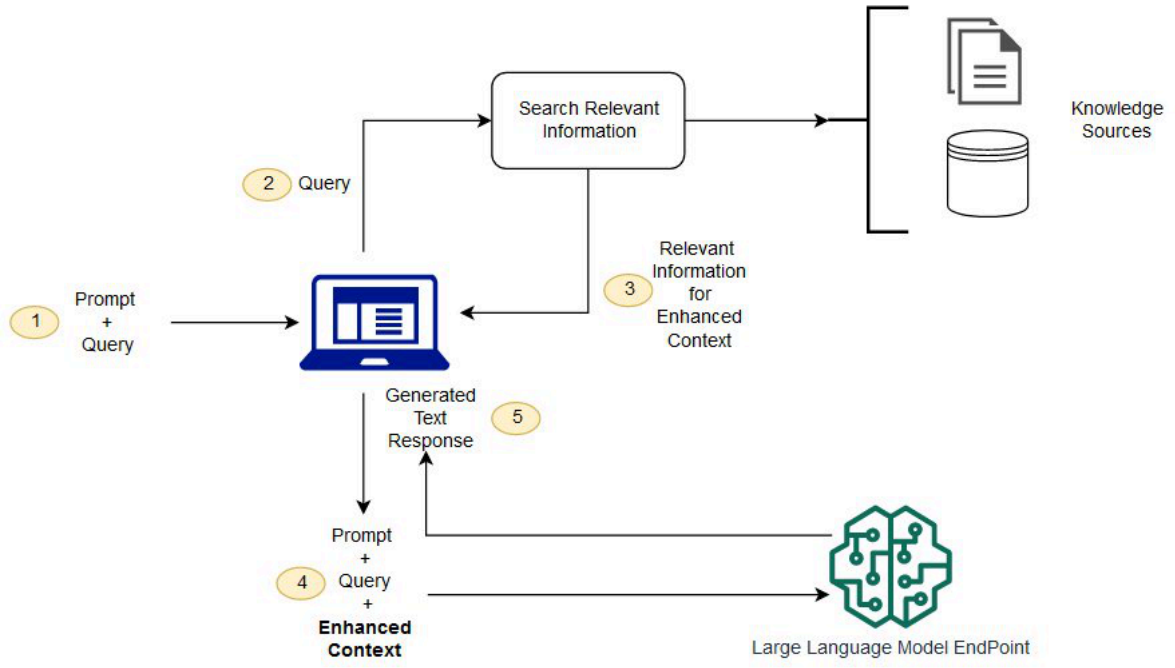


Figure 22: An example of integrating RAG with LLMs for a question answering application, showcasing the process of query extraction, information retrieval, and response synthesis [17].

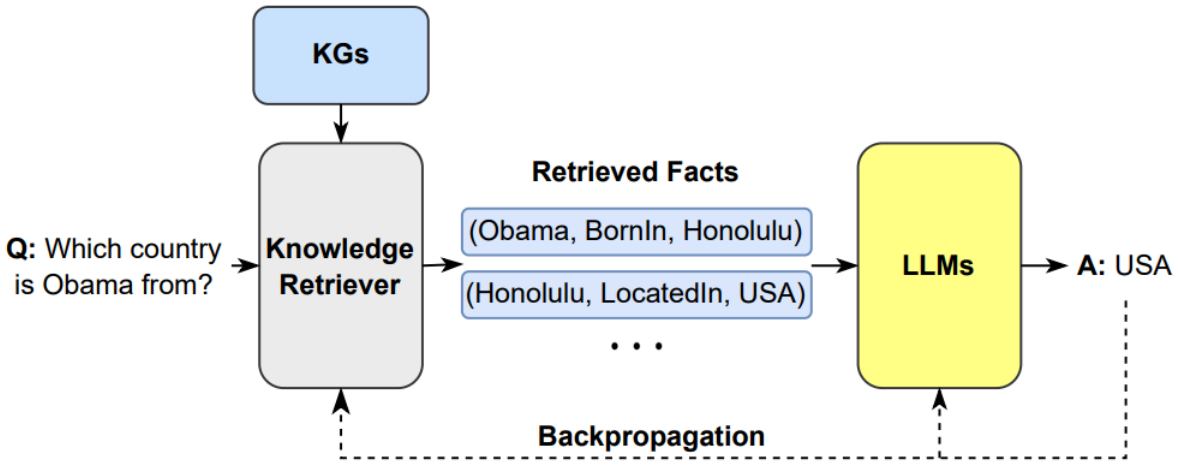


Figure 23: Illustration of using a Knowledge Graph (KG) as a retrieval mechanism in conjunction with LLMs to enhance response generation with structured external knowledge [18].

RAG operates by formulating queries from input prompts and leveraging these queries to fetch pertinent information from diverse sources, such as search engines (see figure 22) or knowledge graphs(see figure 23). This retrieved content is seamlessly integrated into the LLM's workflow, significantly augmenting its ability to generate informed and contextually relevant responses.

5.1 RAG-aware Prompting Techniques

The advent of RAG has spurred the development of sophisticated prompting techniques designed to leverage its capabilities fully. Among these, Forward-looking Active Retrieval Augmented Generation (FLARE) stands out for its innovative approach to enhancing LLM performance.

FLARE iteratively enhances LLM outputs by predicting potential content and using these predictions to guide information retrieval. Unlike traditional RAG models, which typically perform a single retrieval step before generation, FLARE engages

in a continuous, dynamic retrieval process, ensuring that each segment of the generated content is supported by the most relevant external information.

This process is characterized by an evaluation of confidence levels for each generated segment. When the confidence falls below a predefined threshold, FLARE prompts the LLM to use the content as a query for additional information retrieval, thereby refining the response with updated or more relevant data.

For a comprehensive understanding of RAG, FLARE, and related methodologies, readers are encouraged to consult the survey on retrieval augmented generation models, which provides an in-depth analysis of their evolution, applications, and impact on the field of LLMs [19].

6 LLM Agents

The concept of AI agents, autonomous entities that perceive, decide, and act within their environments, has evolved significantly with the advent of Large Language Models (LLMs). LLM-based agents represent a specialized instantiation of augmented LLMs, designed to perform complex tasks autonomously, often surpassing simple response generation by incorporating decision-making and tool utilization capabilities.

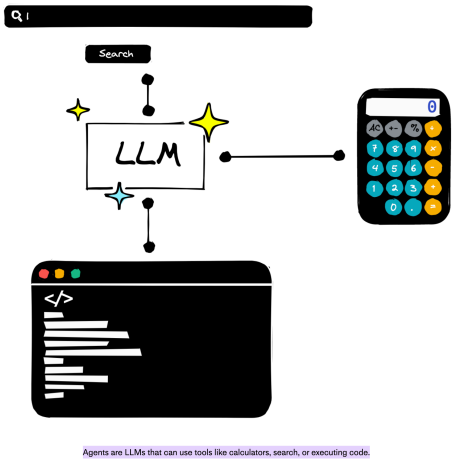


Figure 24: Example block representation of an LLM-based agent, highlighting its components and their interaction in task execution.

LLM agents can access external tools and services, leveraging them to complete tasks, and making informed decisions based on contextual input and predefined goals. Such agents can, for instance, interact with APIs to fetch weather information or execute purchases, thereby acting on the external world as well as interpreting it.

6.1 Prompt Engineering Techniques for Agents

The integration of LLMs into agent frameworks has led to the development of novel prompt engineering techniques, including Reasoning without Observation (ReWOO), Reason and Act (ReAct), and Dialog-Enabled Resolving Agents (DERA), each tailored to enhance the autonomous functionality of LLM-based agents.

6.1.1 Reasoning without Observation (ReWOO)

ReWOO enables LLMs to construct reasoning plans without immediate access to external data, relying instead on a structured reasoning framework that can be executed once relevant data becomes available (see figure 25). This approach is

particularly useful in scenarios where data retrieval is costly or uncertain, allowing LLMs to maintain efficiency and reliability.

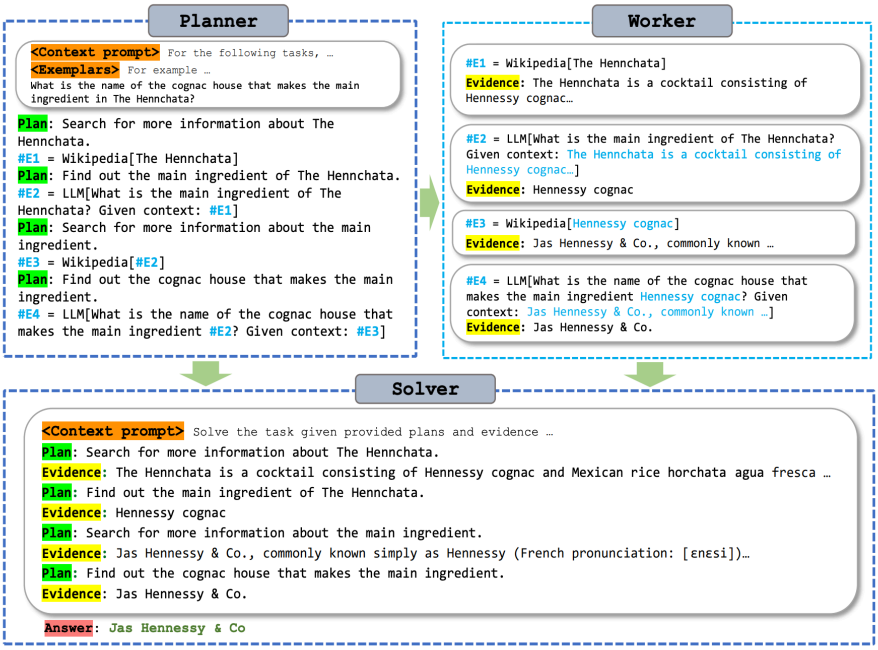


Figure 1: Workflow of ReWOO . Given a question, Planner composes a comprehensive blueprint of interlinked plans prior to tool response. The blueprint instructs Worker to use external tools and collect evidence. Finally, plans and evidence are paired and fed to Solver for the answer.

Figure 25: Workflow of ReWOO, illustrating the meta-planning and execution phases in the reasoning process.

6.1.2 Reason and Act (ReAct)

ReAct (see figure 26) enhances LLMs' problem-solving capabilities by interleaving reasoning traces with actionable steps, facilitating a dynamic approach to task resolution where reasoning and action are closely integrated.

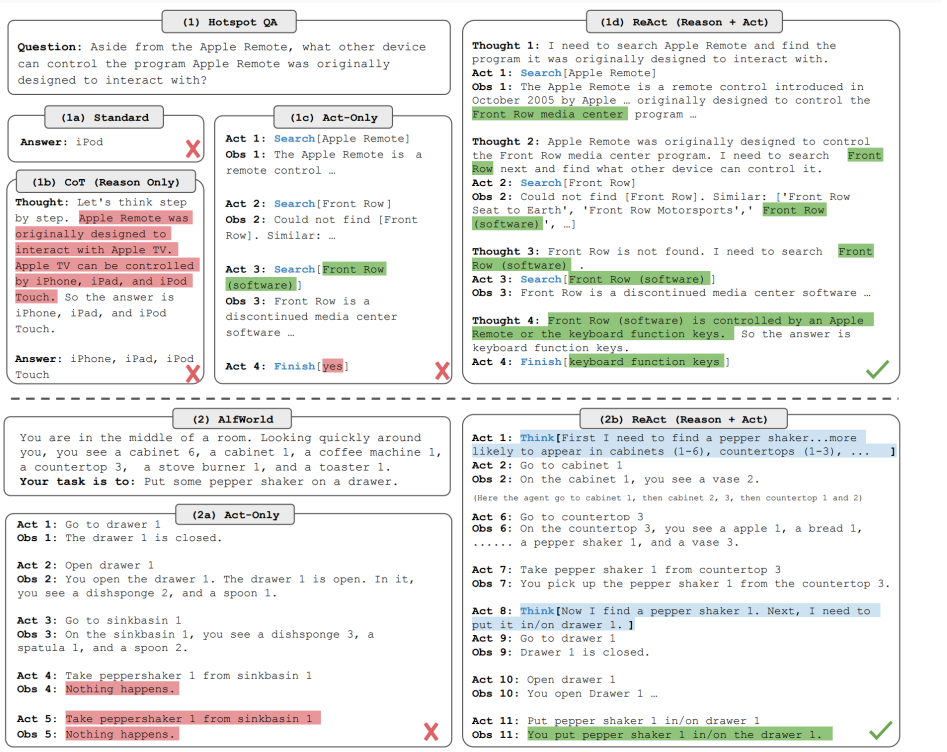


Figure 1: (1) Comparison of 4 prompting methods, (a) `Standard`, (b) Chain-of-thought (`CoT`, Reason Only), (c) `Act`-only, and (d) `ReAct` (Reason+Act), solving a HotpotQA (Yang et al., 2018) question. (2) Comparison of (a) `Act`-only and (b) `ReAct` prompting to solve an AlfWorld (Shridhar et al., 2020b) game. In both domains, we omit in-context examples in the prompt, and only show task solving trajectories generated by the model (Act, Thought) and the environment (Obs).

Figure 26: Comparison of ReAct with simpler prompting methods, highlighting its interleaved reasoning-action structure.

6.2 Dialog-Enabled Resolving Agents (DERA)

DERA (see figure 27) introduces a collaborative agent framework where multiple agents, each with specific roles, engage in dialogue to resolve queries and make decisions. This multi-agent approach enables handling complex queries with depth and nuance, closely mirroring human decision-making processes.

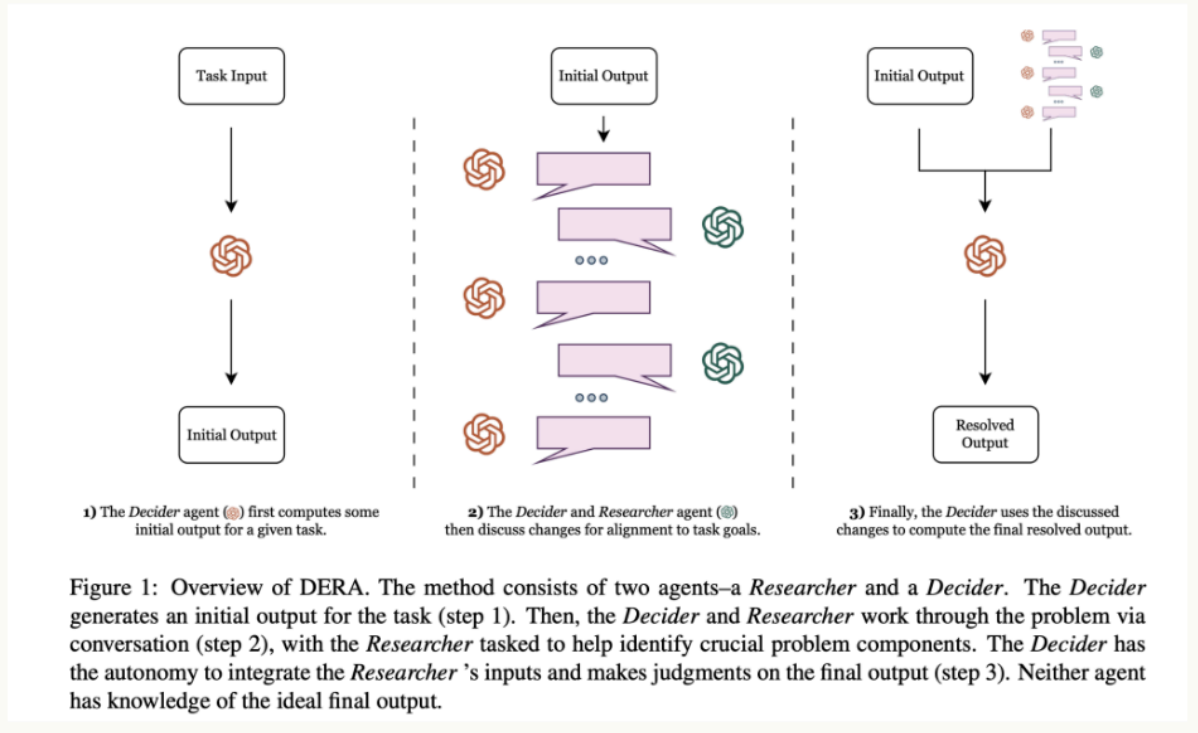


Figure 1: Overview of DERA. The method consists of two agents–a *Researcher* and a *Decider*. The *Decider* generates an initial output for the task (step 1). Then, the *Decider* and *Researcher* work through the problem via conversation (step 2), with the *Researcher* tasked to help identify crucial problem components. The *Decider* has the autonomy to integrate the *Researcher* 's inputs and makes judgments on the final output (step 3). Neither agent has knowledge of the ideal final output.

Figure 27: Conceptual representation of DERA, showcasing the interaction between different agent roles within a dialogue context.

The development of LLM-based agents and associated prompt engineering techniques represents a significant leap forward in AI, promising to enhance the autonomy, decision-making, and interactive capabilities of LLMs across a wide range of applications.

7 Prompt Engineering Tools and Frameworks

The proliferation of advanced prompt engineering techniques has catalyzed the development of an array of tools and frameworks, each designed to streamline the implementation and enhance the capabilities of these methodologies. These resources are pivotal in bridging the gap between theoretical approaches and practi-

cal applications, enabling researchers and practitioners to leverage prompt engineering more effectively.

Langchain has emerged as a cornerstone in the prompt engineering toolkit landscape, initially focusing on Chains but expanding to support a broader range of functionalities including Agents and web browsing capabilities. Its comprehensive suite of features makes it an invaluable resource for developing complex LLM applications.

Semantic Kernel, by Microsoft, offers a robust toolkit for skill development and planning, extending its utility to include chaining, indexing, and memory access. Its versatility in supporting multiple programming languages enhances its appeal to a wide user base.

The **Guidance** library, also from Microsoft, introduces a modern templating language tailored for prompt engineering, offering solutions that are aligned with the latest advancements in the field. Its focus on modern techniques makes it a go-to resource for cutting-edge prompt engineering applications.

Nemo Guardrails by NVidia is specifically designed to construct Rails, ensuring that LLMs operate within predefined guidelines, thereby enhancing the safety and reliability of LLM outputs.

LlamaIndex specializes in data management for LLM applications, providing essential tools for handling the influx of data that these models require, streamlining the data integration process.

From Intel, **FastRAG** extends the basic RAG approach with advanced implementations, aligning closely with the sophisticated techniques discussed in this guide,

and offering optimized solutions for retrieval-augmented tasks.

Auto-GPT stands out for its focus on designing LLM agents, simplifying the development of complex AI agents with its user-friendly interface and comprehensive features. Similarly, **AutoGen** by Microsoft has gained traction for its capabilities in agent and multi-agent system design, further enriching the ecosystem of tools available for prompt engineering.

These tools and frameworks are instrumental in the ongoing evolution of prompt engineering, offering a range of solutions from foundational prompt management to the construction of intricate AI agents. As the field continues to expand, the development of new tools and the enhancement of existing ones will remain critical in unlocking the full potential of LLMs in a variety of applications.

8 Conclusion

Prompt design and engineering will only become more critical as LLMs and generative AI evolve. We discussed foundations and cutting-edge approaches such as Retrieval Augmented Generation (RAG) – essential tools for the next wave of intelligent applications. As prompt design and engineering rapidly progress, resources like this will offer a historical lens on early techniques. Remember, innovations like Automatic Prompt Engineering (APE) covered here could become standard practice in the years to come. Be part of shaping the trajectory of these exciting developments!

References

[1] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, and Michael Young.
Machine learning: The high interest credit card of technical debt.

In *SE4ML: Software Engineering for Machine Learning (NIPS 2014 Workshop)*, 2014.

[2] Xavier Amatriain, Ananth Sankar, Jie Bing, Praveen Kumar Bodigutla, Timothy J. Hazen, and Michaeel Kazi.
Transformer models: an introduction and catalog, 2023.

[3] Xavier Amatriain.
Measuring and mitigating hallucinations in large language models: A multifaceted approach, 2024.

[4] Hattie Zhou, Azade Nova, Hugo Larochelle, Aaron Courville, Behnam Neyshabur, and Hanie Sedghi.
Teaching algorithmic reasoning via in-context learning, 2022.

[5] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp.
Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity, 2022.

[6] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou.
Chain-of-thought prompting elicits reasoning in large language models.
In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc., 2022.

[7] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola.
Automatic chain of thought prompting in large language models, 2022.

[8] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan.
Tree of thoughts: Deliberate problem solving with large language models, 2023.

[9] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom.
Toolformer: Language models can teach themselves to use tools, 2023.

[10] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez.
Gorilla: Large language model connected with massive apis, 2023.

[11] Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro.
Art: Automatic multi-step reasoning and tool-use for large language models, 2023.

[12] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales.
Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models, 2023.

[13] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao.
Reflexion: Language agents with verbal reinforcement learning, 2023.

[14] Sarah J. Zhang, Samuel Florin, Ariel N. Lee, Eamon Niknafs, Andrei Marginean, Annie Wang, Keith Tyser, Zad Chin, Yann Hicke, Nikhil Singh, Madeleine Udell, Yoon Kim, Tonio Buonassisi, Armando Solar-Lezama, and Iddo Drori.
Exploring the mit mathematics and eecs curriculum using large language models, 2023.

[15] Tongshuang Wu, Ellen Jiang, Aaron Donsbach, Jeff Gray, Alejandra Molina, Michael Terry, and Carrie J Cai.
Promptchainer: Chaining large language model prompts through visual programming, 2022.

[16] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba.
Large language models are human-level prompt engineers, 2023.

[17] Amazon Web Services.
Question answering using retrieval augmented generation with foundation models in amazon sagemaker jumpstart, 2023.

[18] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu.
Unifying large language models and knowledge graphs: A roadmap.
arXiv preprint arXiv:2306.08302, 2023.

[19] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang.
Retrieval-augmented generation for large language models: A survey.

arXiv preprint arXiv:2312.10997, 2023.

Experimental support, please view the build logs for errors. Generated by LaTeXML (https://math.nist.gov/~BMiller/LaTeXML/).

Instructions for reporting errors

We are continuing to improve HTML versions of papers, and your feedback helps enhance accessibility and mobile support. To report errors in the HTML that will help us improve conversion and rendering, choose any of the methods listed below:

- Click the "Report Issue" (🐞) button, located in the page header.

Tip: You can select the relevant text first, to include it in your report.

Our team has already identified the following issues (https://github.com/arXiv/html_feedback/issues). We appreciate your time reviewing and reporting rendering errors we may not have found yet. Your efforts will help us improve the HTML versions for all readers, because disability should not be a barrier to accessing research. Thank you for your continued support in championing open access for all.

Have a free development cycle? Help support accessibility at arXiv! Our collaborators at LaTeXML maintain a list of packages that need conversion (https://github.com/brucemiller/LaTeXML/wiki/Porting-LaTeX-packages-for-LaTeXML), and welcome developer contributions (https://github.com/brucemiller/LaTeXML/issues).